



Tarea n°1
“Laberinto Saltarín”
Inteligencia Artificial (2025-1)

Alumna:

Constanza Fabiola Cristinich Ananias

Profesor:

Julio Godoy del Campo

Concepción, Bío-Bío, Abril del 2025.

1.Introducción

La presente tarea consiste en la implementación del “Laberinto Saltarín”, definido como una grilla $m \times n$ de números saltarines, una celda inicial y una celda de destino. Según el número en cada celda, los únicos movimientos permitidos son avanzar dicho número de manera vertical o horizontal tal que, no se cambie la dirección del avance a mitad del camino y no se termine fuera de los límites del laberinto, el objetivo es encontrar el camino más corto partiendo desde el inicio hacia la meta o destino.

Además del desarrollo del juego, se crean tres agentes capaces de cumplir con el objetivo principal del Laberinto mediante tres distintos métodos; ***DFS o búsqueda por profundidad, costo uniforme con costo constante y variable.***

2.Implementación

La implementación del laberinto y de los agentes fue desarrollada con ***Python 3.10.12*** y ***PyGame 2.6.0***, se separa la explicación de la implementación por sus áreas principales. Todo lo explicado a continuación puede encontrarse tanto en el archivo .zip entregado, como en el [link adjunto](#).

2.1 Lectura:

- a) **leer_laberintos(nombre_archivo):** Función que recibe el nombre del archivo que contiene el input y extrae los datos entregados por este para crear una lista con los laberintos como diccionario.
- b) **imprimir_laberintos(laberintos):** Función que recibe la lista de laberintos y la imprime, usada más que nada para confirmar que la lista haya sido creada de manera correcta.
- c) **verificar_archivo(nombre_archivo):** Función que recibe el nombre del archivo con el input, verifica que este archivo exista y que pueda abrirse, si no, envía un mensaje de error.

2.2 Agentes:

- a) **dfs(laberinto):** Función que recibe un laberinto representado como un diccionario y aplica el algoritmo de búsqueda en profundidad (DFS) para encontrar un camino desde la posición inicial hasta la meta. Usa una pila para explorar las posiciones posibles, considerando los saltos permitidos en cada celda en las cuatro direcciones. Al encontrar la meta, actualiza el camino más corto disponible. Devuelve el camino más corto encontrado o ***None*** si no existe ruta.
- b) **costo_uniforme(laberinto):** Función que recibe un laberinto representado como un diccionario y aplica el algoritmo de búsqueda de costo uniforme. Utiliza una cola de prioridad para explorar los caminos desde la posición inicial hasta la meta, priorizando los caminos de menor costo. Cada movimiento entre celdas tiene un costo de 1, y se

consideran movimientos en las cuatro direcciones posibles según el salto permitido en cada celda. Devuelve el primer camino encontrado a la meta o *None* si no existe ruta.

- c) *costo uniforme variable(laberinto):* Función que recibe un laberinto representado como un diccionario y aplica una búsqueda de costo uniforme, donde el costo de cada movimiento corresponde al valor del salto realizado (el número de casillas avanzadas). Utiliza una cola de prioridad para explorar los caminos posibles desde la posición inicial hasta la meta, eligiendo siempre el camino de menor costo acumulado. Considera saltos en las cuatro direcciones y devuelve el primer camino encontrado o *None* si no hay solución.

2.3 Visual:

- a) *calcular tam celda(m, n):* Función que calcula el tamaño óptimo de cada celda del laberinto basado en el número de filas (m) y columnas (n), ajustándose al tamaño máximo permitido para que el laberinto se vea bien centrado y proporcional en pantalla.
- b) *dibujar botones(pantalla, fuente):* Función que dibuja los botones de navegación y de selección de agente en la pantalla especificada. Ajusta el tamaño de los botones, cambia su color al pasar el mouse sobre ellos, y dibuja su texto centrado.
- c) *dibujar laberinto(pantalla, laberinto, fuente, camino=None, lab index=None, pasos mostrados=0):* Función que dibuja el laberinto en la pantalla, mostrando también el título con el número del laberinto, los números de salto en cada celda y, si existe, un camino recorrido hasta cierto paso. Resalta el inicio y el destino con los colores rojo y lila respectivamente.
- d) *dibujar camino(pantalla, camino, offset x, offset y, tam celda, pasos mostrados):* Función que dibuja visualmente el contorno del camino recorrido en el laberinto hasta un número específico de pasos mostrados, resaltando las casillas que forman parte de la solución.
- e) *juego(nombre archivo):* Función que ensambla el resto de funciones para que aparezca el laberinto en pantalla y define las acciones de los botones.

3.Input / Output

Incluido en el archivo .zip se encuentra input.txt que ha sido usado para probar el correcto funcionamiento de la implementación, dentro de este se encuentran 6 laberintos distintos. Aquí se desglosan 2 laberintos: Uno exitoso y el otro donde no se encuentra solución.

3.1 Primer laberinto:

Input: 3 3 0 0 2 2

1 1 1

1 2 1

1 1 1

Output: 4

Al presionar el botón de DFS

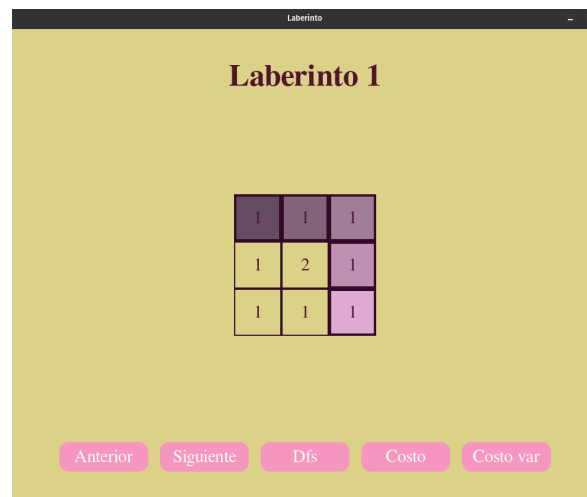
4

Al presionar el botón de Costo uniforme constante

4

Al presionar el botón de Costo uniforme Variable

Interfaz:



3.2 Segundo laberinto:

Input: 4 4 0 0 3 3

2 2 2 2

2 2 2 2

2 2 2 2

2 2 2 2

0

Output: No hay solución usando DFS.

No hay solución usando Costo Uniforme.

No hay solución usando Costo variable.

Interfaz:

